PUCE

Programación Móvil

Semana 3

TEMAS

1. OOP en Kotlin: Evolución del Modelo de Datos.
2. Programación Funcional.
3. Análisis de Adopción Funcional en Proyectos

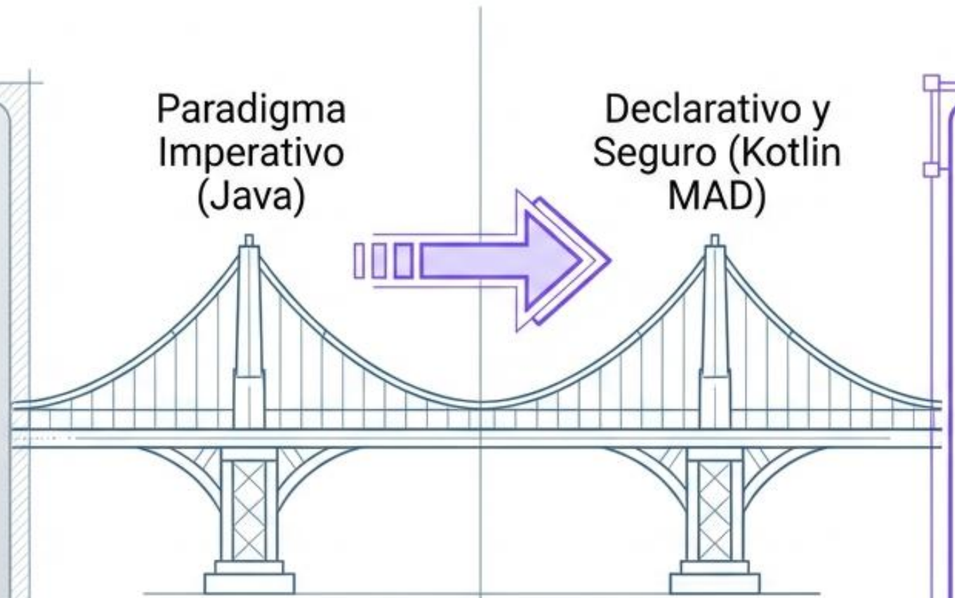
OBJETIVOS

1. Comprender y aplicar data classes, sealed classes y objects para modelar estados complejos en aplicaciones móviles.
2. Transicionar del pensamiento puramente imperativo al funcional utilizando Lambdas y Funciones de Orden Superior.
3. Dominar el uso idiomático de las Scope Functions (let, run, with, apply, also) para manipular objetos de forma segura y concisa.

Introducción: El Cambio de Paradigma Móvil

El Pasado

La arquitectura en Java requería un esfuerzo manual considerable para tareas simples, generando ruido sintáctico (código repetitivo) y exponiendo las aplicaciones al infame NullPointerException.



El Presente

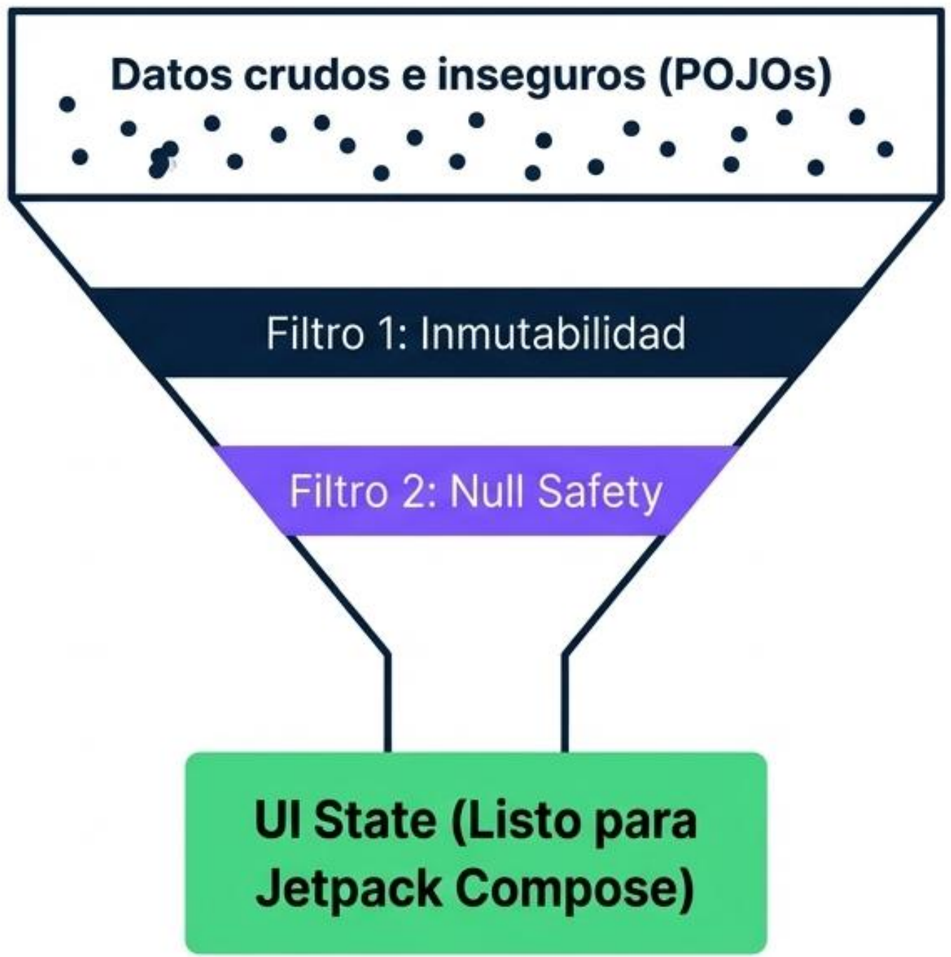
Kotlin no es solo un cambio de sintaxis; es una fusión de la Programación Orientada a Objetos (OOP) con la Programación Funcional (FP). Busca sistemas más predecibles, seguros en tiempo de compilación y altamente mantenibles.

Evolución del **Modelo de Datos** (OOP Avanzada)

- APIs de Red
- Base de datos
- Cualquier Fuente externa

El paradigma orientado a objetos en Kotlin evoluciona para alinearse con la computación moderna. El objetivo es resolver los problemas históricos de Java mediante tres pilares arquitectónicos:

- **Reducción de boilerplate**: Menos líneas de código, menor superficie de errores.
- **Seguridad frente a nulos**: Protección nativa garantizada por el compilador K2.
- **Expresividad**: Código que documenta claramente la intención del desarrollador.



Evolución del modelo de datos: De la **Verbosidad a la Expresividad**

```java
public class User {
    private int id;
    private String name;

    public User(int id, String name) {
        this.id = id;
        this.name = name;
    }
    // + Getters, Setters...
    // + equals()...
    // + hashCode()...
    // + toString()... (¡Más de 40 líneas!)
}
```

Kotlin

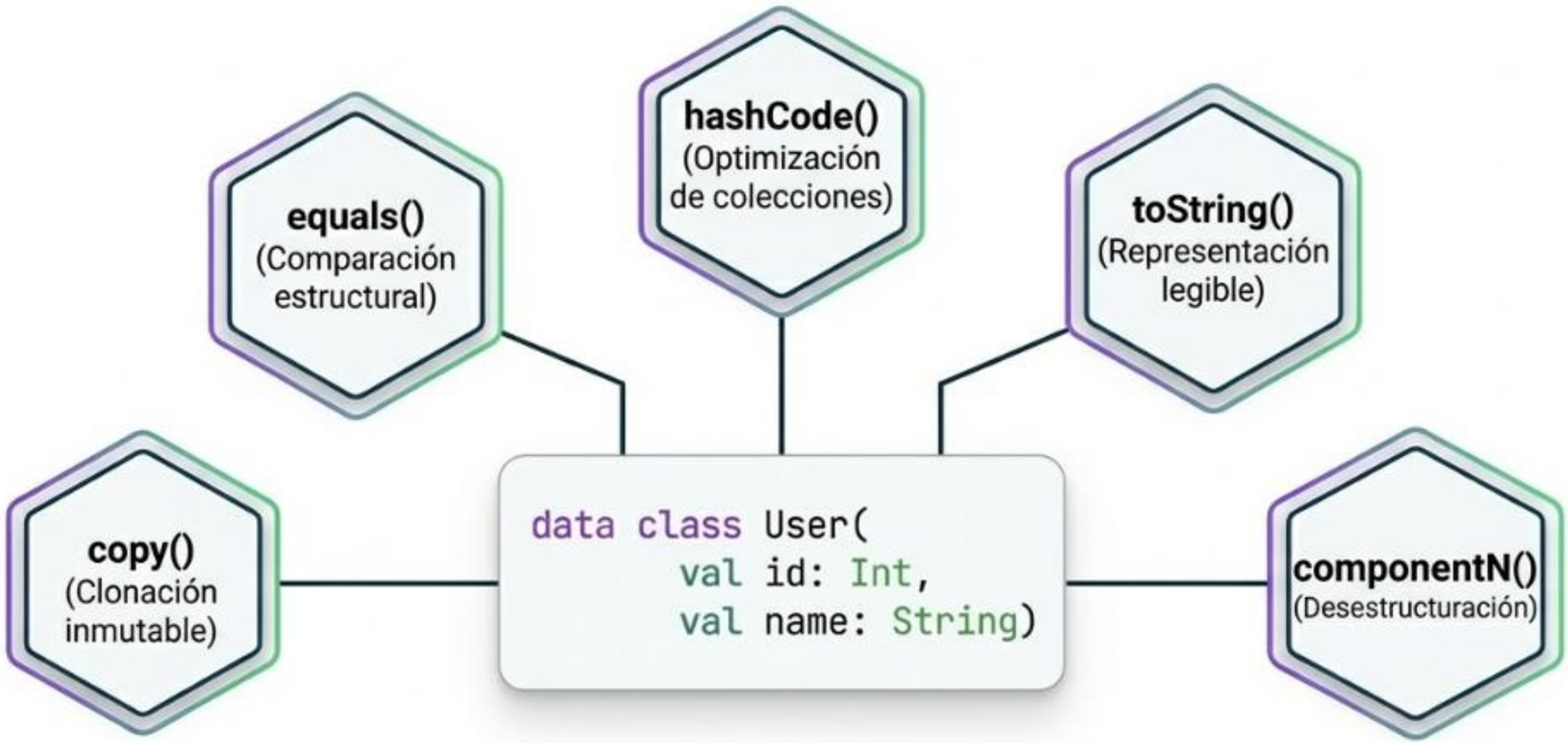
```kotlin
data class User(
    val id: Int,
    val name: String
)
```

- **El Problema Histórico:** Java requiere **decenas de líneas de código repetitivo** (*getters, setters, equals*) para modelar datos simples, aumentando la deuda técnica.
- **La Solución Kotlin:** Sintaxis declarativa que automatiza la estructura, integrando la **seguridad frente a nulos** (Null Safety) directamente en el sistema de tipos.

Kotlin traslada la detección de errores del teléfono del usuario final al escritorio del desarrollador.

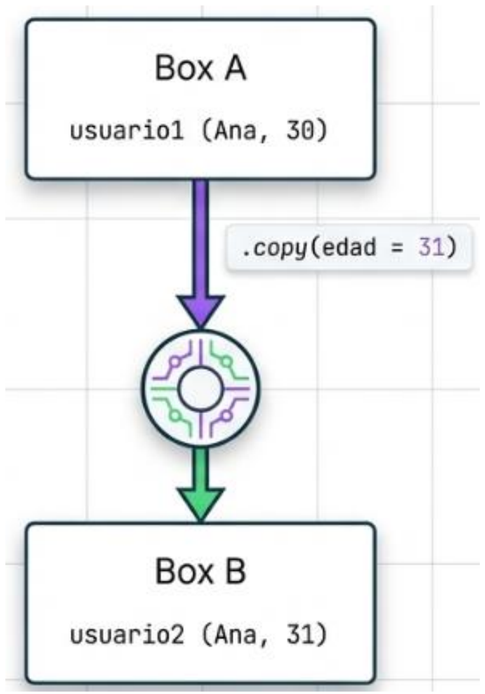
La Solución: Kotlin Data Classes

El compilador de Kotlin asume el trabajo manual. Al añadir la palabra ‘data’, se autogeneran los métodos equals(), hashCode(), toString(), copy() y componentN(). Reglas: Solo las propiedades del constructor primario participan; no pueden ser abstract, open, sealed, ni inner.



Data Clases en Acción: copy() y Desestructuración

En arquitecturas modernas, favorecemos la inmutabilidad. La función `copy()` permite crear una variante exacta de un objeto alterando solo las propiedades necesarias, manteniendo el original intacto. Ideal para Room, DTOs y UI State en Jetpack Compose.




```kotlin
val usuario1 = Usuario("Ana", 30)
val usuario2 = usuario1.copy(edad = 31)

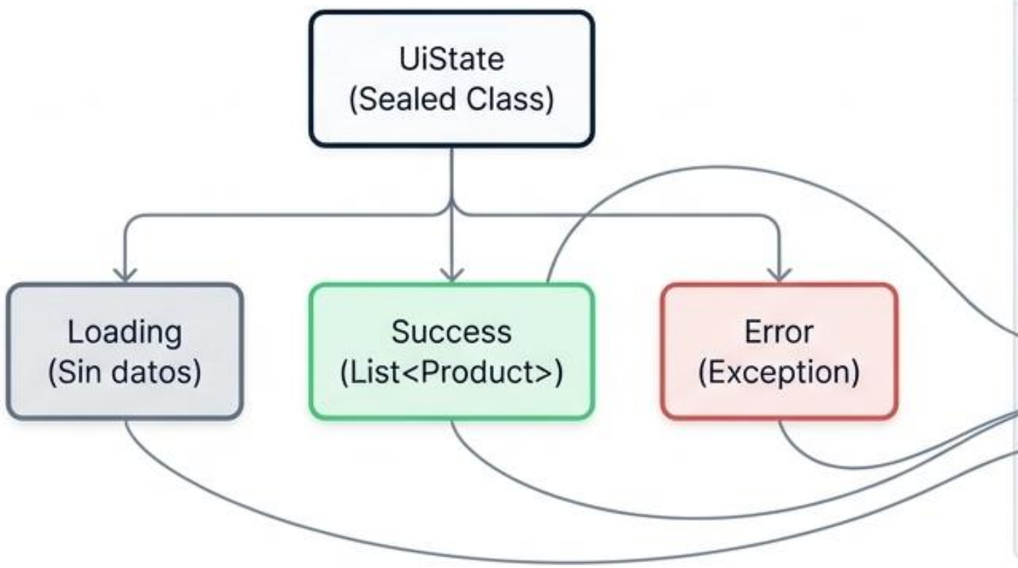
// Desestructuración
val (nombre, edad) = usuario2
println("$nombre tiene $edad años") // Ana tiene 31 años

// Uso típico en Jetpack Compose (API 36)
val user1 = User(1, "Ana")
val user2 = user1.copy(name = "Ana María") // Clonación inmutable rápida
```

En sistemas reactivos (Jetpack Compose), favorecemos la creación de nuevas instancias con propiedades modificadas en lugar de mutar el objeto original (**var**). La función **copy()** evita condiciones de carrera y efectos secundarios indeseados.

Sealed Classes: Jerarquías Cerradas y Máquinas de Estado

A diferencia de un `enum`, las subclases de una clase sellada pueden tener múltiples instancias y diferentes tipos de datos. El compilador conoce todas las variaciones, forzando la exhaustividad en expresiones `when`.




```kotlin
sealed class UiState {
    data object Loading : UiState()
    data class Success(val products: List<Product>) : UiState()
    data class Error(val exception: Exception) : UiState()
}

@Composable
fun ProductScreen(state: UiState) {
    // El compilador nos obliga a manejar todos los casos
    when (state) {
        is UiState.Loading -> CircularProgressIndicator()
        is UiState.Success -> ProductList(state.products)
        is UiState.Error -> ErrorView(state.exception.message)
    }
}
```

Representan jerarquías restringidas. El compilador conoce de antemano todas las posibles subclases, funcionando como interruptores lógicos finitos.

Sealed Classes: Modelado de Estados y Sealed Interfaces

Todas las subclases deben declararse en el mismo archivo o módulo. Desde Kotlin 1.5, las `sealed interfaces` permiten jerarquías más flexibles, vitales para modelar comportamientos transversales y respuestas complejas de API.

Modelado de Estado de Pantalla

```
sealed class LoginUiState {
    object Cargando : LoginUiState()
    data class Exito(val usuario: Usuario)
: LoginUiState()
    data class Error(val mensaje: String)
: LoginUiState()
}
```

Sealed Interfaces (Jerarquías Transversales)

```
sealed interface Error
sealed interface NetworkError : Error

data class ServerError(val mensaje:
String) : NetworkError
data class ConnectionError(val mensaje:
String) : NetworkError
```

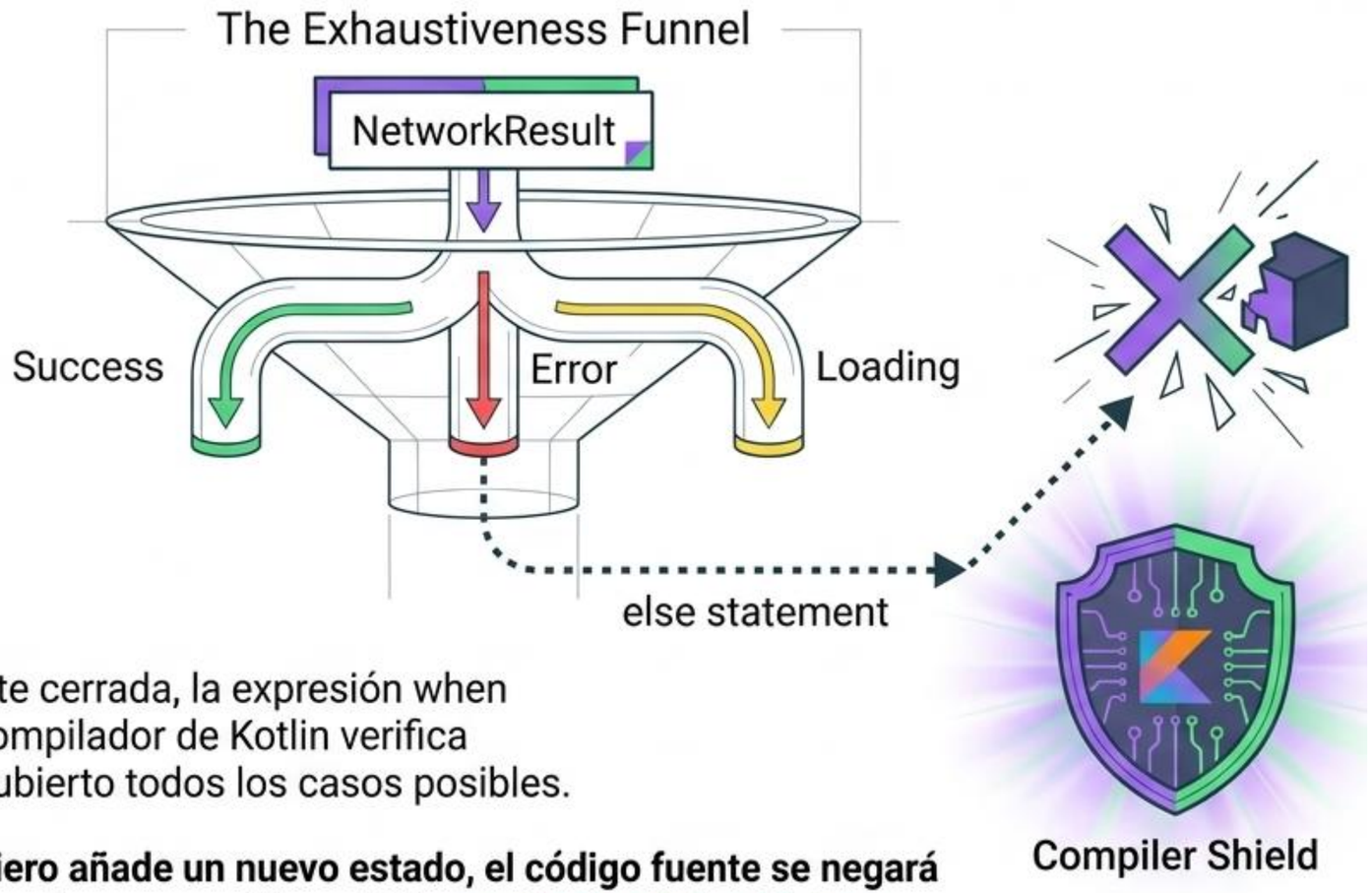
Sealed class → "todos estos tipos son hijos de la misma familia"

```
sealed class Animal
class Perro : Animal()
class Gato : Animal()
class Loro : Animal()
```

Sealed interface → "todos estos tipos cumplen el mismo rol, aunque no sean familia"

```
sealed interface Volador
class Avion : Volador
class Murcielago : Volador
class Drone : Volador
class SuperMan : Volador
```

El Escudo del Compilador: Exhaustividad del `when`



Al ser una jerarquía estrictamente cerrada, la expresión when actúa de forma exhaustiva. El compilador de Kotlin verifica matemáticamente que se han cubierto todos los casos posibles.

Impacto: **Si en el futuro un ingeniero añade un nuevo estado, el código fuente se negará a compilar en todos los lugares donde no se gestione. Esto erradica los fallos lógicos en producción.**

El Escudo del Compilador: Exhaustividad del `when` (ejemplo UBER)

Supongamos que tenemos:

```
sealed class TripState {
    data object Searching : TripState()
    data class DriverFound(val driver: Driver) : TripState()
    data class Arriving(val eta: Int) : TripState()
    data class InProgress(val route: Route) : TripState()
    data object Completed : TripState()
}
```

Y mañana se agrega:

```
data object Cancelled : TripState()
```

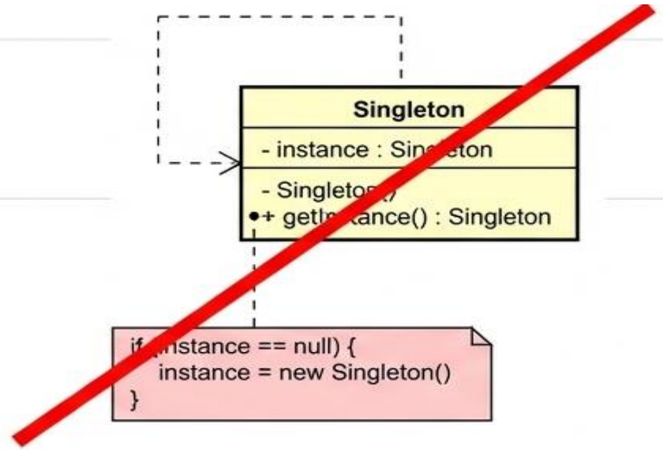
Todas las pantallas que tenían:

```
when(state) {
    is Searching -> …
    is DriverFound -> …
    is Arriving -> …
    is InProgress -> …
    is Completed -> …
}
```

Ahora fallan al compilar. ¿Por qué?

object:

- Sirve para modelar **entidades globales del Sistema** (bases de datos, cliente HTTP, repositorios)
- El Patrón Singleton Nativo – adios 'static'



Concepto: Kotlin elimina los miembros estáticos. La palabra clave object define una clase y crea su única instancia simultáneamente.

El Dolor de Java: Lograr un Singleton seguro (Double-Checked Locking, Bill Pugh) requería código defensivo complejo para evitar problemas multihilo.

La Solución Kotlin: Seguridad de hilos (thread-safe) garantizada por el lenguaje mediante inicialización perezosa (lazy).

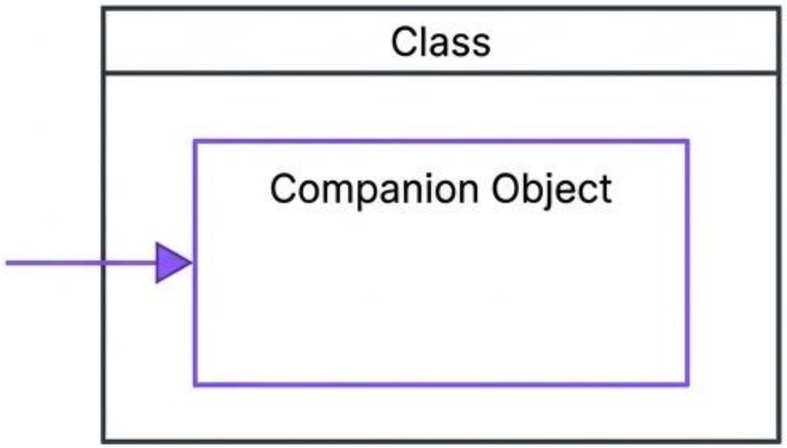
```kotlin
object DatabaseConfig {
    const val DB_NAME = "app_database.db"

    fun connect() {
        /* lógica */
    }
}

// Uso: DatabaseConfig.connect()
```

Característica	Java (Tradicional)	Kotlin (Idiomático)
Declaración	Constructor privado + método getInstance()	`object Usuario { ... }`
Gestión de Hilos	Lógica de sincronización manual requerida	Seguro por defecto (Thread-safe nativo)

Companion objects: Fábricas y Lógica de Clase



Concepto: El reemplazo idiomático para static en Java. Es un objeto real (puede implementar interfaces) que vive dentro de una clase.

Casos de uso:
- Métodos de fábrica (Factories)
- Constantes asociadas
- Validaciones previas a la instanciación

```kotlin
class User private constructor(val id: String) {
    companion object {
        fun fromEmail(email: String): User {
            require(email.contains("@"))
            return User(email)
        }
    }
}
```

Ejemplo válido:

```kotlin
val user = User.fromEmail("juan@puce.edu.ec")
```

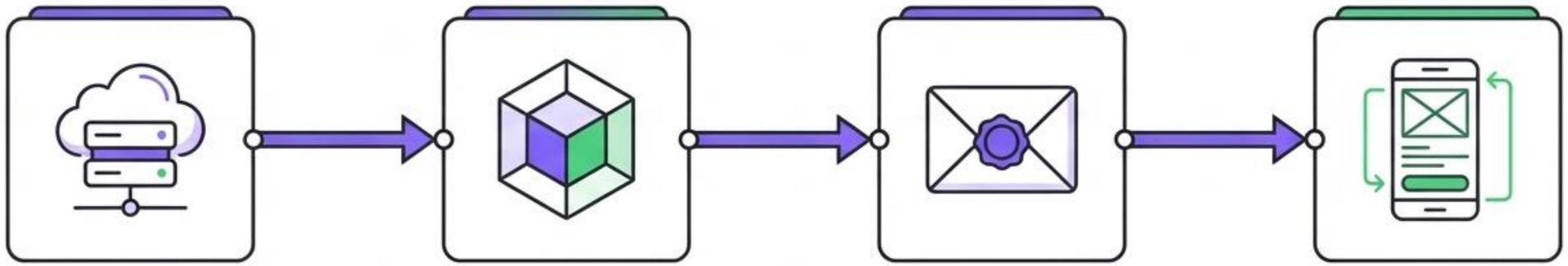
Ejemplo inválido:

```kotlin
User("cualquier cosa")   // ❌ No compila
User("juan")             // ❌ Lanza excepción
```

Nota Arquitectónica:
Permite mantener el constructor primario privado, forzando a crear instancias solo a través de vías validadas por el companion object.

A diferencia de los bloques estáticos de Java, un companion object es un objeto real: retiene estado, puede implementar interfaces y centraliza la lógica de validación previa a la instanciación.

Síntesis: El Ecosistema Kotlin en Acción



Paso 1: Petición (Object)

Una instancia única (object ApiClient) ejecuta la llamada a la red de forma segura.

Paso 2: Modelo de Datos (Data Class)

La red devuelve un paquete JSON parseado automáticamente a una estructura inmutable (data class User).

Paso 3: Máquina de Estado (Sealed Class)

Ese User se envuelve en un estado estricto: UiState.Success(user).

Paso 4: Interfaz Reactiva (Compose)

La interfaz evalúa el UiState con un when exhaustivo y renderiza la pantalla sin riesgo de nulos.

Estas herramientas no son elementos aislados; son engranajes diseñados para encajar perfectamente y eliminar clases enteras de errores antes de compilar la app.

Síntesis: El Ecosistema Kotlin en Acción (Ejemplo Uber)

Paso 1: Petición (object)

El cliente de red (API) obtiene la ubicación del conductor:

```
object TripApi {
    suspend fun getDriverLocation(): DriverLocationResponse { ... }
}
```

Paso 2: Modelo de Datos (data class)

La respuesta se convierte en un modelo inmutable:

```
data class DriverLocation(
    val lat: Double,
    val lng: Double,
    val etaMinutes: Int
)
```

Paso 3: Máquina de Estado (sealed class)

El ViewModel envuelve ese dato en un estado:

```
sealed class TripUiState {
    data object Loading : TripUiState()
    data class DriverNearby(val location: DriverLocation) : TripUiState()
    data class Error(val message: String) : TripUiState()
}
```

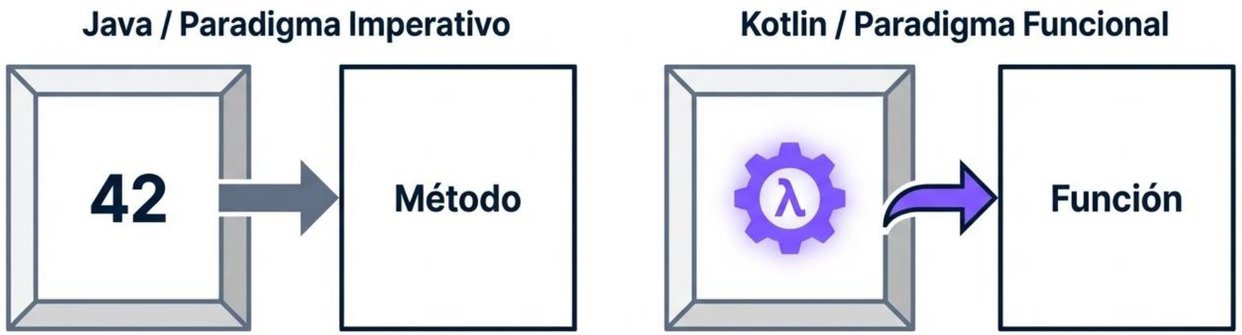
Paso 4: Interfaz Reactiva (Compose)

La UI reacciona automáticamente:

```
@Composable
fun TripScreen(state: TripUiState) {
    when (state) {
        is TripUiState.Loading -> LoadingView()
        is TripUiState.DriverNearby -> MapView(state.location)
        is TripUiState.Error -> ErrorView(state.message)
    }
}
```

Programación Funcional: Una herramienta para manejar la complejidad

- Kotlin integra la **programación funcional (FP)** como complemento de la OOP.
- En FP no solo pasamos datos a funciones: **pasamos funciones como datos**.
- **Jetpack Compose** está construido completamente sobre este paradigma.
- Las funciones son **ciudadanos de primera clase**:
  - Se asignan a variables
  - Se pasan como parámetros
  - Se retornan desde otras funciones

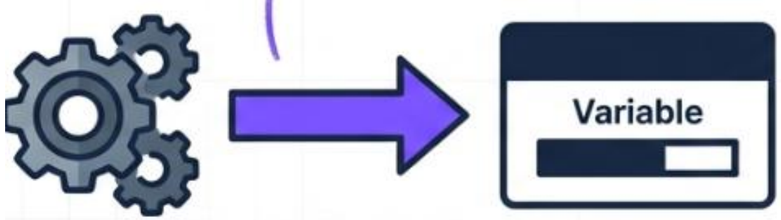



```kotlin
fun operar(x: Int, operacion: (Int) -> Int): Int {
    return operacion(x)
}

val resultado = operar(5) { n -> n * 2 }
```

Lamdas: Ciudadanos de Primera Clase

Las expresiones lambda son literales de función. Su diseño en Kotlin permite código extremadamente limpio gracias a convenciones como el 'trailing lambda' y el parámetro implícito `it`. Además, soportan clausuras, permitiendo capturar y modificar variables del contexto externo.



En Kotlin, las funciones no están atadas a una clase. Pueden guardarse en variables, pasarse como parámetros o devolverse como resultados, permitiendo arquitecturas altamente dinámicas.

```
// Sintaxis Básica: Un literal de función entre llaves { }
val cuadrado = { x: Int -> x * x }

println(cuadrado(5)) // 25
```

Inicio de la lambda

Separación entre parámetros y cuerpo

VS

Java

```
class Matematica {
    int cuadrado(int x) {
        return x * x;
    }
}

Matematica m = new Matematica();
m.cuadrado(5);
```

Lamdas: Sintaxis y Uso Idiomático

🔧 Sintaxis básica de lambdas

```
val cuadrado = { x: Int -> x * x }

println(cuadrado(5)) // 25
```

🧩 Lambdas con `it` (parámetro implícito)

- Si la lambda tiene un solo parámetro, Kotlin infiere la variable **`it`** implícitamente reduciendo drásticamente el ruido visual:

```
val lista = listOf(1, 2, 3) // // Uso idiomático de it con colecciones

val cuadrados = lista.map { it * it }
```

✨ Trailing Lambda

- Si la lambda es el **último parámetro**, puede ir fuera de los paréntesis.
- Facilita la lectura del código.

Clausuras: Capturando el Contexto

🔒 Lambdas que recuerdan valores externos

Las lambdas pueden acceder y modificar variables fuera de su propio ámbito.

🧠 Ejemplo con clausura

```
var factor = 2
val multiplicar = { x: Int -> x * factor }
println(multiplicar(4)) // 8
factor = 3
println(multiplicar(4)) // 12
```

📌 ¿Por qué es importante?

- Permite crear funciones altamente configurables.
- Útil en Compose, callbacks, filtros, transformaciones y lógica reactiva.

Beneficios de las Lambdas en Kotlin

🚀 Ventajas clave

- Código más expresivo y conciso
- Menos clases anónimas verbosas (como en Java antiguo)
- Facilita la composición de funciones
- Ideal para trabajar con colecciones, flujos y UI declarativa

```
lista
    .filter { it > 0 }
    .map { it * 2 }
    .sorted()
```

Antes (Java)

```
lista.stream().map(new Function<Integer, Integer>() {
    @Override
    public Integer apply(Integer x) {
        return x * x;
    }
});
```

Ahora (Kotlin)

```
lista.map { it * it }
```

Funciones que Reciben Funciones: El Siguiente Nivel en Kotlin

🔍 ¿Qué es una Función de Orden Superior?

- Es una función que recibe otra función como parámetro o devuelve una función.
- Permiten abstraer patrones comunes y escribir código más expresivo.
- En Android, reemplazan objetos anónimos con → ahora solo pasamos una lambda.

🏵️ Ejemplo básico

```
fun operar(a: Int, b: Int, operacion: (Int, Int) -> Int): Int {    return
  operacion(a, b)
}

val suma = operar(4, 5) { x, y -> x + y }

println(suma) // 9

val resta = operar(10, 3) { x, y -> x - y }

val multiplicacion = operar(6, 7) { x, y -> x * y }

val maximo = operar(8, 2) { x, y -> maxOf(x, y) }
```

Lambda como parámetro
Paso de un comportamiento

Lambdas + Orden Superior: La Cocina del Código

Una función de orden superior acepta lambdas como parámetros o devuelve una función.
Analogía: El método es el Chef. No le entregas comida preparada (datos); le entregas una receta (lambda) que le dicta cómo procesar los ingredientes.




```kotlin
// Función de Orden Superior: recibe una operación matemática como parámetro
fun calcular(a: Int, b: Int, operacion: (Int, Int) -> Int): Int {
    return operacion(a, b)
}

// Usando lambdas para definir la operación
val suma = calcular(5, 3, { x, y -> x + y })

// Sintaxis idiomática en Kotlin (Trailing Lambda):
// Si la lambda es el último parámetro, puede ir fuera de los paréntesis
val multiplicacion = calcular(5, 3) { x, y ->
    x * y
}

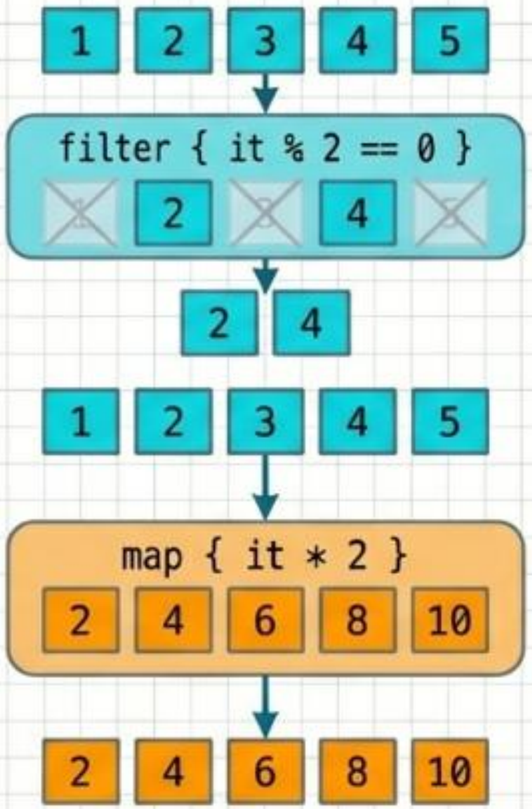
// Esto es exactamente cómo funciona un Botón en Compose:
Button(onClick = { guardarDatos() }) {
    Text("Guardar")
}
```

Definición de la fun Button

```kotlin
@Composable
fun Button(
    onClick: () -> Unit,
    content: @Composable () -> Unit
)
```

Aplicación de Orden Superior: Transformación de Datos

En lugar de iterar manualmente con bucles (imperativo), declaramos qué queremos hacer con cada elemento de forma declarativa y funcional.



Paradigma Declarativo
(Qué quiero obtener?)

```
val numeros = listOf(1, 2, 3, 4, 5)
val pares = numeros.filter { it % 2 == 0 }
```

Función de Orden Superior -> Recibe un lambda como argumento

```
val dobles = numeros.map { it * 2 }
```

💡 **Nota:** Operaciones adicionales como `reduce` permiten consolidar colecciones en un solo valor acumulado siguiendo el mismo paradigma declarativo puro.

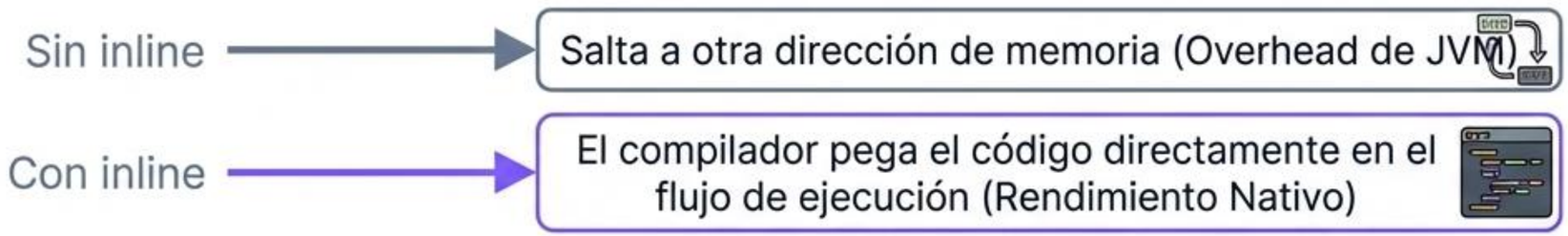
VS

Paradigma Imperativo
(Cómo hacerlo?)

```
val pares = mutableListOf<Int>()
for (n in numeros) {
    if (n % 2 == 0) {
        pares.add(n)
    }
}
```

Inline Functions: Rendimiento sin Overhead

Las funciones de orden superior introducen una ligera penalización al crear objetos de función en la JVM. La palabra clave `inline` instruye al compilador a copiar el cuerpo de la función directamente en el sitio de llamada, eliminando la asignación de memoria.
Limitaciones: No aplicable a funciones recursivas.




```kotlin
inline fun medirTiempo(accion: () -> Unit) {
    val inicio = System.currentTimeMillis()
    accion()
    val fin = System.currentTimeMillis()
    println("Tiempo: ${fin - inicio} ms")
}

// El cuerpo de 'medirTiempo' y el println se compilan directamente aquí
medirTiempo { println("Ejecutando proceso pesado...") }
```

Extension Functions: Qué son?

Es una forma de añadir nuevas funciones a una clase sin modificarla, sin heredarla y sin usar patrones externos como utilidades estáticas.

✨ Potenciando APIs existentes en Kotlin

- El compilador las traduce a **métodos estáticos en Java**, donde el primer parámetro es el *receiver*.
- Se resuelven **estáticamente** → no hay polimorfismo dinámico como en métodos miembros.

🔑 Beneficios idiomáticos

- **Claridad:** `file.readFile()` en lugar de `FileUtils.readFile(file)`
- **Seguridad:** extensiones sobre tipos nulables (`String?.isNullOrBlank()`)

Extension Function

```
fun File.readFile(): String {
    return this.readText()
}

val file = File("datos.txt")
file.readFile()
```

Receiver

Extension Functions: Ejemplos

🧩 Ejemplo básico

```
fun String.esEmailValido(): Boolean =
    contains("@") && contains(".")

val email = "juan@empresa.com"
println(email.esEmailValido()) // true
```

🧠 Puente Cognitivo

- En Java usarías `StringUtils.isValidEmail(email)`
- En Kotlin, parece un método propio del objeto:

```
fun String.isEmailValid(): Boolean {
    return this.contains("@") && this.contains(".")
}
fun main() {
    val correo = "estudiante@universidad.edu"
    if (correo.isEmailValid()) {
       println("Válido")
 }
}
```

📌 Usos comunes

- Añadir utilidades a tipos estándar
- Mejorar legibilidad y expresividad



Extension Properties y Buenas Prácticas

🏷️ Extension Properties

```
val String.dominioEmail: String
     get() = substringAfter("@")
val correo = "juan@puce.edu.ec"
println(correo.dominioEmail)
```

🧭 Cuándo crearlas

- Cuando la función opera principalmente sobre un tipo específico
- Para evitar clases utilitarias estáticas
- Para lograr sintaxis más natural y expresiva

✔️ Buenas prácticas

- Usarlas libremente, pero **controlar visibilidad**
- Preferir extensiones **locales o privadas** si no son de uso general
- No usarlas para lógica compleja o dependiente de miembros privados

🎯 Beneficios finales

- Fomentan composición y reutilización
- Código más idiomático y legible
- Integración fluida con librerías de terceros

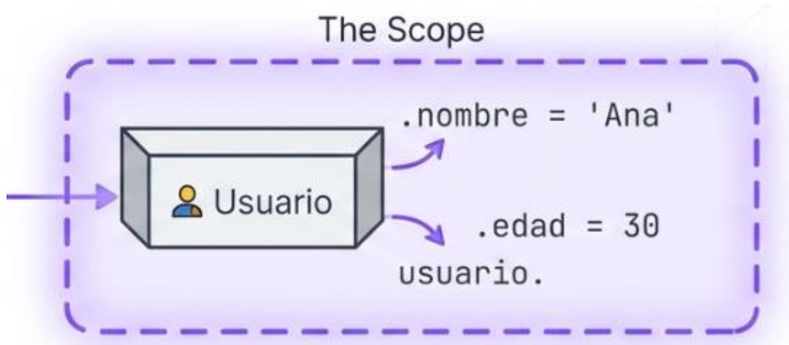
¿Qué son las Scope Functions en Kotlin?

🎯 Idea central

Las *scope functions* (`let`, `run`, `with`, `apply`, `also`) permiten ejecutar un bloque de código **dentro del contexto de un objeto**, creando un **ámbito temporal** que evita repetir su nombre.

🧩 ¿Por qué existen?

- Reducen código repetitivo.
- Mejoran la legibilidad.
- Permiten configurar objetos de forma más limpia.



Ejemplo de Scope Function adecuada para evitar repetición: apply

💡 **Problema original:** El código repite innecesariamente el nombre del objeto:

```
usuario.nombre = "Ana"
usuario.edad = 30
usuario.validar()
usuario.imprimir()
```

🎯 Solución idiomática en Kotlin: apply

- Usa `this` como contexto implícito
- Permite configurar un objeto sin repetir su nombre
- Retorna **el mismo objeto**, ideal para inicialización y encadenamiento
- Expresa la intención de "configurar este objeto" de forma declarativa

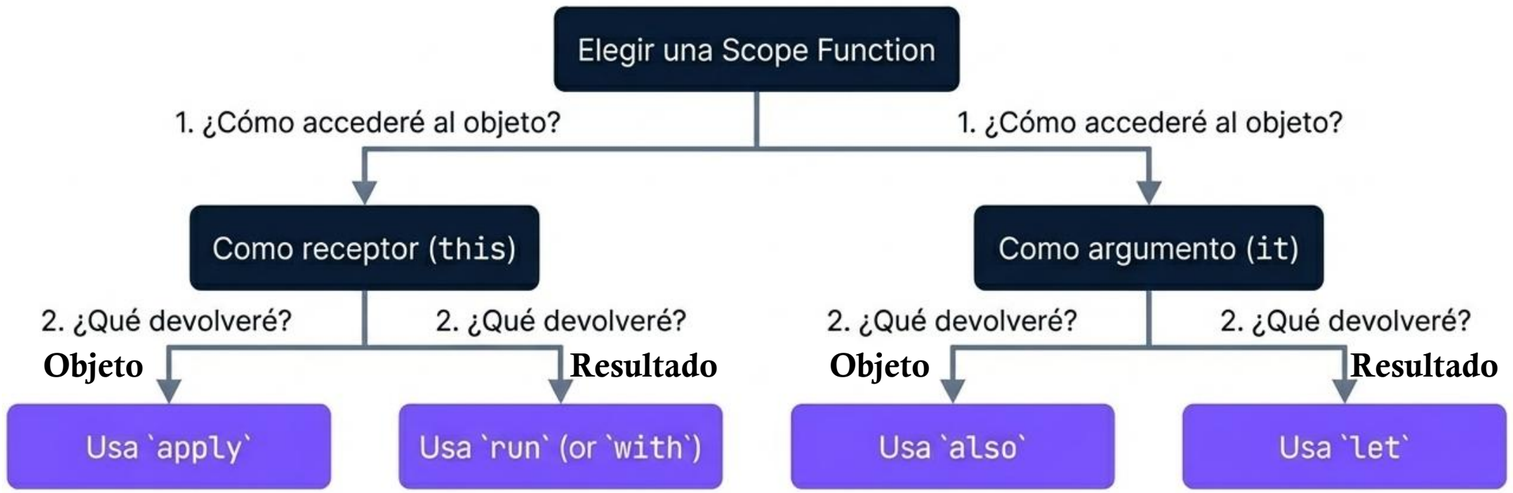
✅ Código refactorizado con apply:

```
usuario.apply {
    nombre = "Ana"
    edad = 30
    validar()
    imprimir()
}
```

¿Cómo elegirla una Scope Function?

🧠 Preguntas clave para elegir la función correcta

1. **¿Cómo quiero referirme al objeto dentro del bloque?**
   - `this` → objeto receptor
   - `it` → argumento lambda
2. **¿Qué quiero que devuelva?**
   - El **objeto original modificado**
   - El **resultado del bloque**



Análisis de las Scope Functions en Kotlin

🎯 **Importancia:** Dominar `let`, `run`, `with`, `apply`, `also` distingue a un desarrollador junior de uno senior en Kotlin. Cada función tiene un propósito semántico específico.

🧩 1. `let`

- Referencia: **it**
- Retorna: **resultado de la lambda**
- Ideal para:
  - Ejecutar código solo si un objeto **no es nulo**
  - Transformar un objeto en otro valor

Ejemplos

```
val nombre: String? = "Ana"
nombre?.let { valor: String ->
   println("El nombre es $it")}


val longitud = nombre?.let { it.length }
```

🧩 2. `apply`

- Referencia: **this**
- Retorna: **el objeto original**
- Uso típico: configuración de objetos sin repetir el nombre

Ejemplos

```
val usuario = Usuario("Ana", 30).apply {
     email = ana@example.com
     telefono =   "0999999999"
     activarNotificaciones()
     validarDatos()
}
```

run, also y with con ejemplos

3. `run`

- Referencia: **this**
- Retorna: **resultado de la lambda**
- Útil para: configurar un objeto y luego **calcular un valor**

Ejemplo

```
data class Usuario(
    val nombre: String,
    val edad: Int
)

val usuario = Usuario("Ana", 30)

val resultado = usuario.run {
    "$nombre tiene $edad años"
}
```

4. `also`

- Recibe el objeto como **argumento (it)**
- Ejecutar un bloque de código que **no modifica el objeto**
- Devuelve el objeto original.
- Uso típico: efectos secundarios (logs, validaciones)

Ejemplo

```
val usuario = Usuario("Ana", 30).also {
    println("Usuario creado: $it")
}
```

5. with

- Referencia: **this**
- No es extensión
- Recibe el **objeto como argumento**
- Útil para agrupar múltiples operaciones sobre un mismo objeto

Ejemplo

```
data class Usuario(
        val nombre: String,
        val edad: Int
)
val usuario = Usuario("Ana", 30)
with(usuario) {
        println(nombre)
        println(edad)
}
```

Matriz de decisión para elegir la Scope Function adecuada

Función	Acceso al contexto	Valor de retorno	Caso de uso común
apply	this	El mismo objeto	Inicialización y configuración
also	it	El mismo objeto	Logging, validaciones, efectos secundarios
let	it	Resultado de la lambda	Null-safety, transformaciones
run	this	Resultado de la lambda	Configuración + cálculo
with	this	Resultado de la lambda	Agrupar llamadas sobre un objeto existente

🎯 **Idea Clave:** a elección correcta depende de cómo accede al objeto (this/it) y qué necesita que retorne (objeto o resultado).

Errores Comunes y Buenas Prácticas

❌ 1. Anidar múltiples `let` o `run`

Esto genera confusión sobre si el contexto actual es `it` o `this`. Ejemplo:

```
usuario?.let {
    it.direccion?.let {
        println(it.calle)
    }
}
```

✓ **Buena práctica: nombrar explícitamente el parámetro**

```
usuario?.let { user ->
    println(user.nombre)
}
```

❌ 2. Usar `apply` para cálculos complejos

`apply` **siempre devuelve el objeto original,** no el resultado del cálculo.

✓ **Regla de oro:**

- Si necesita **modificar un objeto y devolverlo →** `apply` o `also`
- Si necesita **obtener un resultado basado en el objeto →** `let`, `run` o `with`

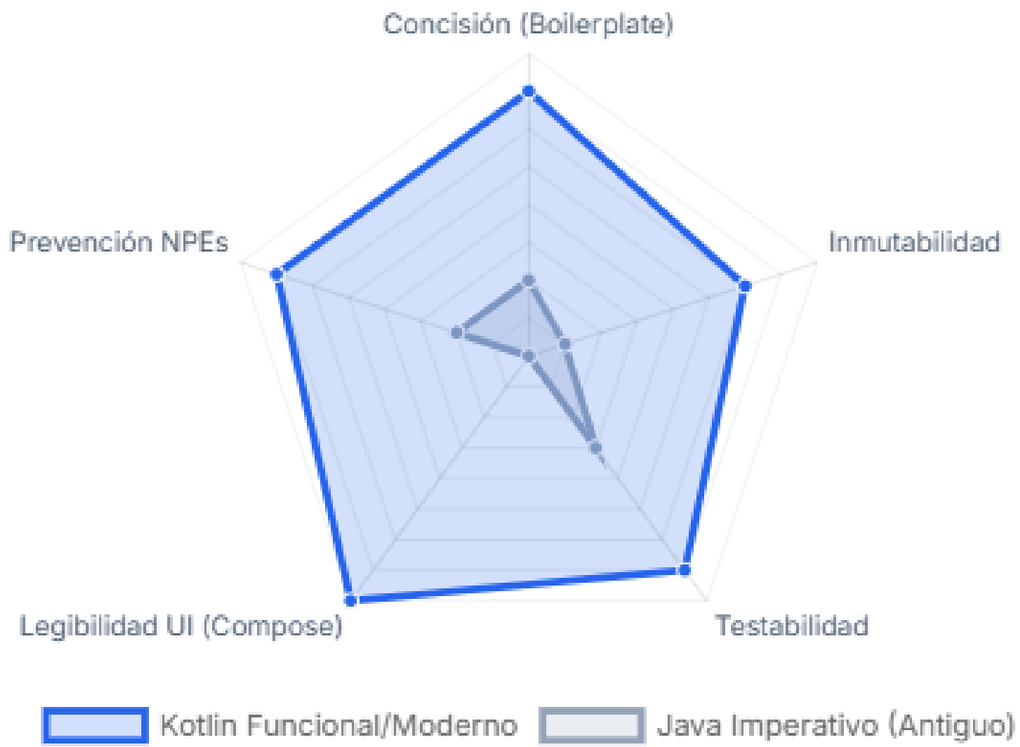
Migración de Imperativo a Funcional en Android Moderno

🎯 Idea central

La adopción de un enfoque **funcional e idiomático** en Kotlin produce mejoras medibles en proyectos Android, especialmente al migrar desde código **imperativo clásico**.

📊 Beneficios observados en la práctica

- **Mayor concisión**: menos código repetitivo y estructuras verbosas.
- **Mejor inmutabilidad**: reducción de estados mutables y efectos colaterales.
- **Incremento en testabilidad**: funciones puras y estructuras predecibles.
- **Legibilidad superior en UI moderna (Compose)**: declaratividad natural.
- **Prevención de NPEs**: gracias al sistema de tipos seguro para nulls.



Ejemplo típico de adopción funcional

🧪 Uso de funciones de alcance para transformar código imperativo en expresiones más limpias:

```
val resultado = usuario?.let { user ->
    "${user.nombre} está activo"
}
```

```
val configurado = Usuario("Ana", 30).apply {
    // Configuración adicional
}
```

Actividad 1: El dilema del estado de Red

🎯 Problema a resolver

Está desarrollando una App que descarga un perfil de usuario. Debe representar **cuatro estados mutuamente excluyentes**:

- **Inactivo**
- **Descargando**
- **Exitoso** (incluye un `User`)
- **Error** (incluye código y mensaje)

❓ Pregunta

¿Qué estructura de Kotlin permite modelar estos estados de forma segura, expresiva y extensible?

✅ Respuesta idiomática

La herramienta ideal es una **sealed class**, ya que permite definir un conjunto cerrado de estados, algunos con datos asociados.

🧪 Ejemplo de solución en Kotlin

```
sealed class NetworkState {
    data object Idle : NetworkState()
    data object Downloading : NetworkState()
    data class Success(val user: User) : NetworkState()
    data class Error(val code: Int, val message: String) : NetworkState()
}
```

🧠 Razón pedagógica

Las *sealed classes* permiten representar **estados finitos**, garantizan **exhaustividad** en `when` y facilitan un manejo seguro del flujo de red.

Actividad 2: Refactorización a Funcional

🎯 Objetivo

Transformar una función imperativa clásica de Java en una versión **idiomática, concisa y funcional** en Kotlin usando **funciones de orden superior**.

❓ Problema original (Java)

Filtrar números pares desde una lista:

```
List filterEvens(List numbers) {
 List evens = new ArrayList<>();
 for (Integer num : numbers) {
     if (num % 2 == 0) evens.add(num);
 }
 return evens;
}
```

✅ Solución idiomática en Kotlin

Kotlin ofrece funciones de extensión sobre colecciones como `filter`, que aceptan lambdas y permiten expresiones más limpias.

✓ Versión funcional (una sola línea)

```
fun filterEvens(numbers: List) = numbers.filter { it % 2 == 0 }
```

🧠 Idea clave

La programación funcional en Kotlin permite escribir código **más declarativo**, **más legible** y **menos propenso a errores**, aprovechando el poder de las colecciones y las lambdas.

Actividad 3: Salvando el NullPointerException

🎯 Objetivo

Manejar correctamente un valor **nullable** (`String?`) proveniente de una API, evitando un `NullPointerException` y ejecutando lógica solo cuando el dato es válido.

❓ Pregunta

¿Qué Scope Function combinada con Safe Call (?.) permite ejecutar código solo si el valor no es nulo?

✅ Solución idiomática

La función **`let`** es el estándar para trabajar con valores opcionales:

- Se ejecuta **solo si el objeto no es nulo**.
- Dentro del bloque, el valor se vuelve **no-null** automáticamente.

🧪 Ejemplo en Kotlin

```
val nombre: String? = api.getName()
nombre?.let { nameSafe ->
 // nameSafe ya no es nullable dentro de este bloque
 println("Longitud: ${nameSafe.length}")
 actualizarUI(nameSafe)
}
```

🧠 Idea clave

`?.let { ... }` es el patrón idiomático para combinar **null-safety**, **transformación segura** y **ejecución condicional** en Kotlin moderno.

Resumen de la Semana

- Hemos aprendido que Kotlin reduce el código redundante de Java (Data Classes)
- Hace los estados más seguros (Sealed Classes) y
- Permite inyectar comportamiento de forma elegante (Extension Functions).